PROJECT REPORT

AI INTERVIEWER

An AI-Powered Technical Interview Practice Web Application

Submitted in partial fulfilment of the requirements for the award of

[Degree / Diploma Name]

Submitted by: [Student Name]

Roll / Registration No.: [Roll Number]

Department: [Department Name]

Institution: [College / University Name]

Academic Year: [Academic Year]

PROJECT GUIDE: [Guide / Supervisor Name]

CERTIFICATE

This is to certify that the project report entitled “AI Interviewer” is a record of the project work carried out by [Student Name], bearing roll or registration number [Roll Number], under my guidance and supervision during the academic year [Academic Year]. The work is submitted to [College / University Name] in partial fulfilment of the requirements for [Degree / Diploma Name].

To the best of my knowledge, the report presents the design, implementation, and documented functionality of an AI-assisted technical interview practice web application. The report has been prepared for academic evaluation. Any external libraries, services, and documentation used in the work are acknowledged in the references section.

Project Guide	Head of Department	External Examiner
Signature: _____	Signature: _____	Signature: _____
Name: _____	Name: _____	Date: _____

Date: _____ Place: _____

STUDENT DECLARATION

I, [Student Name], declare that the project entitled “AI Interviewer” and the accompanying report have been prepared by me as part of the academic requirements of [Degree / Diploma Name] at [College / University Name]. The report describes the project implementation and its functionality to the best of my understanding.

I have cited the technologies, libraries, services, and documentation that informed the work. I understand that the AI Interviewer application uses third-party artificial-intelligence and transcription services, and that their outputs are generated estimates rather than guaranteed assessments. I have not intentionally included passwords, API keys, or other private credentials in this report.

I understand that the final report must be reviewed and completed with accurate student, course, department, and guide information before submission. Screenshots included here are illustrative captures of the application's public interface; they do not claim to show private user records.

Student name: [Student Name]

Roll / registration number: [Roll Number]

Signature: _____

Date and place: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to [Project Guide / Supervisor Name] for guidance, feedback, and encouragement during the planning and development of AI Interviewer. The suggestions provided during the project helped me organize the requirements, understand the application workflow, and prepare this report in a clear academic format.

I am thankful to the faculty members and staff of [Department Name], [College / University Name], for providing the learning environment and resources needed to complete this mini project. I also appreciate the support of my classmates, friends, and family, whose discussions and encouragement helped me continue through implementation and documentation.

I acknowledge the maintainers of the open-source frameworks and libraries used in the project, including React, Vite, Redux Toolkit, Express, Mongoose, FastAPI, and related packages. I also acknowledge the documentation and provider services that support authentication, question generation, and audio transcription. Their use is described in the technology and references sections.

Finally, I thank everyone who gave feedback on the interface and the project demonstration. Any errors or omissions remaining in this report are my responsibility.

With gratitude, [Student Name]

ABSTRACT

AI Interviewer is a full-stack web application designed to help learners rehearse technical interviews through a structured practice-and-review workflow. After signing in, a user chooses a target role, seniority level, interview format, and number of questions. The system creates a session and requests role-oriented questions from an AI service. During the interview, the user can submit a spoken answer recorded through the browser or provide code through an embedded editor. Oral audio is uploaded, validated, converted where needed, and transcribed. The answer is then evaluated and stored with technical and confidence scores, written feedback, and an ideal-answer reference.

The frontend is implemented with React and Vite and uses React Router and Redux Toolkit for navigation and state. A Node.js and Express backend provides authentication and session APIs, protects user-specific records, and coordinates background work. MongoDB stores user accounts, session state, questions, submissions, and evaluation results. A FastAPI AI microservice and backend integrations connect the application to Gemini and AssemblyAI; FFmpeg supports audio conversion. Socket.IO communicates generation and evaluation progress.

The completed-session report presents summary metrics, a question-level performance chart, and detailed answer feedback. The application also provides responsive navigation and four persistent accent themes. The project demonstrates practical integration across web UI, authentication, persistence, asynchronous APIs, audio handling, AI services, and deployment. AI feedback is intended as a learning aid, not a verified hiring decision. Future work includes automated end-to-end tests, queue-based processing, longitudinal analytics, and a mock mode.

Keywords: AI interview practice, React, Express, MongoDB, FastAPI, voice transcription, technical assessment, full-stack application.

TABLE OF CONTENTS

S. No.	Chapter / Section	Page No.
1	Certificate	2
2	Student Declaration	3
3	Acknowledgement	4
4	Abstract	5
5	List of Figures	8
6	List of Tables	9
7	Abbreviations and Terms	10
8	Introduction to the Project	11
9	Background and Motivation	12
10	Problem Statement	13
11	Aim, Objectives, and Success Criteria	14
12	Project Scope and Assumptions	15
13	Development Methodology	16
14	Requirement Analysis	17
15	Functional Requirements in Detail	18
16	Non-Functional Requirements	19
17	System Architecture Overview	20
18	Frontend Design and Navigation	21
19	Backend API and Request Processing	22
20	Database Design and Data Dictionary	23
21	Authentication and Profile Management	24
22	Dashboard and Session Configuration	26

TABLE OF CONTENTS (CONTINUED)

S. No.	Chapter / Section	Page No.
23	Question Generation Workflow	27
24	Voice Interview and Audio Processing	28
25	Coding Interview Workspace	29
26	AI Evaluation and Scoring	30
27	Session State and Real-Time Updates	31
28	Session Review and Progress Analytics	32
29	Interface Design, Themes, and Accessibility	33
30	Security, Privacy, and Data Handling	34
31	Error Handling and Reliability	35
32	Testing Strategy and Test Cases	36
33	Test Case Table and Expected Results	37
34	User Manual: Account and Session Setup	38
35	User Manual: Answering and Reviewing	39
36	Installation and Configuration	40
37	Deployment and Operations	41
38	Limitations and Known Constraints	42
39	Future Scope and Enhancements	43
40	Conclusion	44
41	References	45
42	Appendix A — Glossary	47
43	Appendix B — Demonstration Checklist	48
44	Appendix C — Project Configuration Checklist	49

LIST OF FIGURES

Fig. No.	Figure Title	Page No.
1	Landing page of AI Interviewer (captured from the running application)	11
2	Login page (captured from the running application)	25
3	Registration page (captured from the running application)	25
4	Layered architecture of the system (conceptual diagram)	20
5	Asynchronous session lifecycle (conceptual flow)	27
6	Audio answer processing pipeline (conceptual flow)	28
7	Session result and analytics layout (conceptual diagram)	32
8	Persistent accent-theme choices (implementation overview)	33

Note: Figures 1–3 are screenshots captured from the running frontend. The remaining figures are explanatory diagrams or implementation summaries prepared for this report, not screenshots of hidden authenticated data.

LIST OF TABLES

Table No.	Table Title	Page No.
1	Technology stack and responsibility	17
2	Functional requirements summary	18
3	Non-functional quality requirements	19
4	User and session data entities	23
5	Interview session states	31
6	Representative system test cases	37
7	Environment variable categories	40
8	Future enhancement roadmap	43

ABBREVIATIONS AND TERMS

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
FFmpeg	Audio and video processing framework used for conversion
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
MERN	MongoDB, Express, React, and Node.js
OAuth	Open Authorization protocol
REST	Representational State Transfer
UI	User Interface
UX	User Experience
WAV	Waveform Audio File Format
WebM	Web media container format
Vite	Frontend development server and build tool

AI Interviewer uses a combination of frontend, backend, database, authentication, audio, and AI terminology. These abbreviations are expanded here for readability. Provider-specific product names retain their conventional capitalization.

CHAPTER 1

Introduction to the Project

Project overview

AI Interviewer is a web application for practicing technical interviews through a guided, repeatable workflow. A signed-in user selects a job role, experience level, interview format, and question count. The system creates a session, presents questions, accepts spoken or code-based answers, and returns AI-generated evaluation. A completed session can then be reviewed question by question. The project is intended as a learning and preparation aid; it does not replace a human interviewer or guarantee a recruitment outcome.

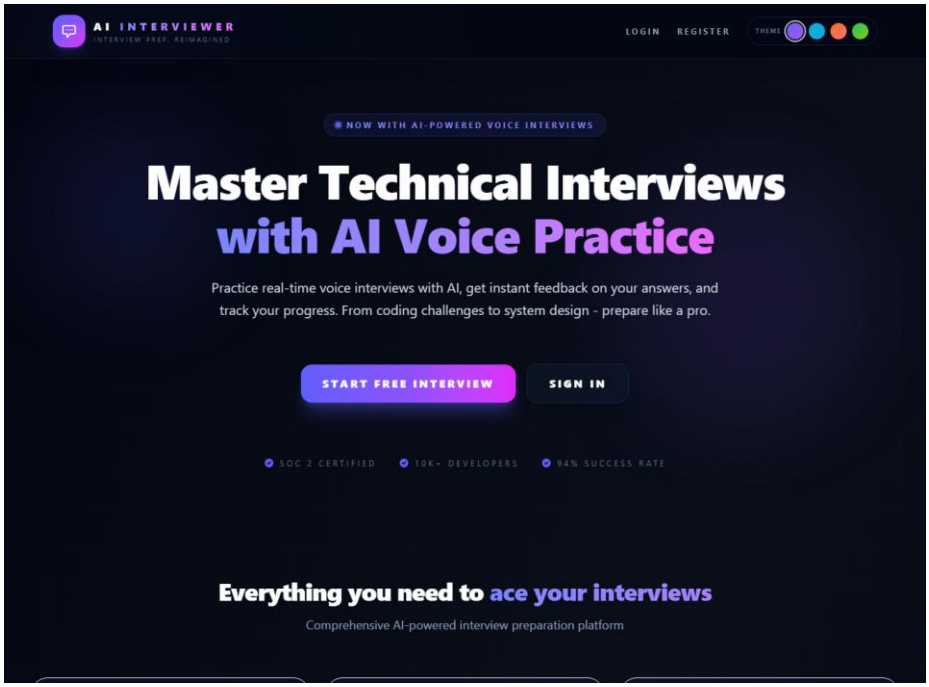


Figure 1. Landing page of AI Interviewer — real application screenshot.

Purpose in an academic context

The project brings together frontend development, REST API design, authentication, persistent data modelling, file upload handling, asynchronous processing, real-time updates, and third-party AI services. It is therefore useful as a full-stack academic mini project: the visible interface can be demonstrated while the underlying components illustrate how separate services cooperate. The application has a deployed frontend and a local development setup documented in the repository README.

CHAPTER 2

Background and Motivation

Interview preparation

Technical interviews assess more than the ability to recall definitions. Candidates may need to reason aloud, explain trade-offs, write code, and organize an answer under time pressure. These skills improve through deliberate repetition. A learner preparing alone may find it difficult to create realistic questions and to receive consistent feedback on every practice attempt.

Need for an accessible practice loop

A web application can make practice available without scheduling a partner. A role and seniority selection makes a session more relevant than a fixed generic quiz. Voice input permits verbal rehearsal, while a code editor supports implementation questions. Recording the result creates a reviewable history, turning an individual session into a learning activity rather than a one-time interaction.

Why use AI in this project

Generative AI can produce a varied set of questions from a role, level, and format prompt and can return structured feedback for a submitted answer. In this application Gemini is used for question generation and evaluation; AssemblyAI is used for audio transcription. The services augment the practice workflow, while the application remains responsible for accounts, session data, validation, routing, and user experience. AI output can be imperfect, so its score is treated as guidance.

CHAPTER 3

Problem Statement

Observed preparation gap

Learners often need to practice several dimensions of interviewing: technical correctness, communication, and confidence. Static study notes provide information, but they do not simulate answering a question. Informal mock interviews can be valuable, yet availability and feedback quality vary. The project addresses this gap by providing an on-demand structured practice environment.

Problem definition

The problem addressed by AI Interviewer is: how can a learner create a role-relevant technical interview session, submit an oral or code answer, and receive a consistent review using a single web application? The system must preserve session progress and distinguish questions that are pending, submitted, evaluated, completed, or failed so the user can understand what is happening.

Practical constraints

The experience depends on network connectivity, configured API credentials, browser microphone support for voice recording, and the availability of external AI and transcription providers. AI-generated content is probabilistic and may require human judgment. The application therefore includes asynchronous status feedback, selected retry paths, input checks, and fallback question generation. These measures improve resilience but cannot guarantee uninterrupted operation of external services.

CHAPTER 4

Aim, Objectives, and Success Criteria

Overall aim

The overall aim is to build a full-stack application that helps users rehearse technical interviews through role-specific questions, spoken or code answers, AI-generated evaluation, and a session review page. The focus is a complete usable workflow rather than a single isolated AI prompt.

Project objectives

The implementation objectives are to provide account registration and login, support Google authentication, protect user-specific routes, let users configure interview sessions, generate tailored questions, record voice answers, accept code submissions in an editor, transcribe and evaluate answers, calculate session-level metrics, and retain a question-by-question review. A further objective is to deploy the frontend and document the backend and AI service configuration.

Observable success criteria

A practical demonstration is successful when a user can authenticate, configure a session, see generated questions, submit at least one supported answer type, observe the evaluation status, finish the session, and inspect its report. Supporting criteria include rejecting invalid or missing submissions, restricting session access to its owner, showing useful progress states, and keeping sensitive environment values out of source-controlled configuration.

CHAPTER 5

Project Scope and Assumptions

Included in scope

The current scope includes a responsive web interface, email/password registration and login, Google sign-in integration, profile and preferred-role settings, session creation and history, question generation, oral-only and coding-mix formats, audio recording and transcription, code submission, AI scoring and feedback, session completion, and review analytics. Four accent palettes—Midnight, Ocean, Sunset, and Forest—are included in the frontend.

Outside the current scope

The repository identifies automated end-to-end testing, background job queues, long-term analytics dashboards, and an AI-quota-free mock mode as future improvements. The current app is not presented as an automated hiring decision system, a real-time human-to-human meeting platform, or a code execution sandbox. The report does not claim that submitted code is executed; the implemented service evaluates the supplied response through the AI evaluation workflow.

Assumptions and dependencies

The user has a modern browser and, for oral practice, grants microphone permission. The deployment has valid database and service credentials. The backend can reach MongoDB, Gemini, and AssemblyAI. Audio upload limits and accepted formats are enforced by backend middleware. Questions, scores, and transcripts are dependent on third-party service availability and the quality of the submitted audio or code.

CHAPTER 6

Development Methodology

Incremental implementation

The project was developed in functional phases: project setup, authentication, interview workspace, voice interviews, AI evaluation, and production deployment. This sequence keeps each stage connected to an observable feature. The repository README records these phases and also lists intended future improvements.

Feature-oriented workflow

A feature begins with a user action, such as selecting a role or submitting an answer. The frontend captures the action and dispatches a Redux async thunk. The Express API validates authorization and request data, updates MongoDB, and calls the appropriate AI service. A response or Socket.IO event updates the interface. This end-to-end view is useful during development because a feature is not complete until its data and user-facing status agree.

Iteration and verification

Development validation includes producing a production frontend build and linting relevant files. The repository does not currently define a complete automated end-to-end test suite, so live demonstrations and focused checks are important. Future work should add repeatable tests for account flows, interview generation, audio submissions, and completed review reports. External API behavior should be isolated behind test doubles where practical.

CHAPTER 7

Requirement Analysis

User roles

The primary user is a learner who creates and reviews their own interview sessions. A visitor can access public landing, login, and registration pages. An authenticated user can access the dashboard, profile, interview, and review routes. The implemented application does not define a separate administrative dashboard in the current frontend route list.

Functional requirements

The system should authenticate users, store a preferred role, create a session from role/level/type/count settings, generate questions, retrieve and delete owned sessions, submit audio or code, evaluate each question, end a session, and calculate review metrics. The user interface should show loading and error states and provide navigation among questions. The interview supports question counts of 5, 10, or 15 and role choices spanning software, data, cloud, security, design, and product work.

Quality requirements

The interface should be responsive and understandable, and routes containing personal session information should require authentication. The API should validate request fields and ownership, place limits on traffic and audio size, and surface processing status. AI calls should handle temporary rate limits where coded. Configuration should be environment-based so that secrets are not embedded into frontend source.

Table 1. Technology stack and responsibility

Layer	Principal technologies	Purpose
Frontend	React, Vite, Redux Toolkit	Screens, routing, state
Backend	Node.js, Express, Socket.IO	API, authorization, updates
Persistence	MongoDB, Mongoose	Users and interview sessions
AI / audio	FastAPI, Gemini, AssemblyAI, FFmpeg	Questions, transcription, evaluation

CHAPTER 8

Functional Requirements in Detail

Account and profile features

A visitor can register with name, email, and password or sign in with credentials. Google OAuth is also supported through a verified ID token. Authenticated users can view and update profile information including preferred role. A JWT is returned by the backend and used to authorize protected API requests. Email is unique in the user model and passwords are hashed before persistence.

Interview session features

An authenticated user can create a session by choosing a role, level, interview type, and question count. The backend initially saves a pending session and starts question creation asynchronously. The user can retrieve the list of their sessions, open an individual session, continue an in-progress session, delete a session, and end a session. Session status values include pending, in-progress, completed, and failed.

Answer and review features

For oral questions, the browser records audio and submits it as a file. For coding questions, a code string is submitted with the question index. The backend transcribes audio when present, evaluates the answer, and stores transcript/code, scores, feedback, and ideal answer. The review screen shows an overall score, average technical and confidence metrics, elapsed session time, charted question scores, and detailed answers.

Table 2. Functional requirement summary

ID	Requirement	Expected outcome
FR-01	Register and authenticate	Valid identity receives authorized access
FR-02	Configure a session	Role, level, format, and count stored
FR-03	Submit oral or code answer	Answer associated with correct question
FR-04	Evaluate and review	Scores and written feedback displayed

CHAPTER 9

Non-Functional Requirements

Usability and accessibility

The app should make the next action clear, show progress through questions, provide explicit recording and submission controls, and present evaluation feedback in a readable structure. Responsive layouts support narrower screens. Focus outlines and reduced-motion handling improve accessibility for keyboard users and people who prefer less animation. Microphone permission is a browser-controlled requirement that should be explained before voice practice.

Performance and responsiveness

The frontend uses lazy-loaded route components for larger pages, reducing the initial bundle work. Question generation and evaluation are asynchronous so the request can return while the AI work continues. Socket.IO communicates status updates to the signed-in user. Backend rate limits and provider retry behavior help control service usage, although exact response time depends on network and AI provider latency.

Security and maintainability

Authentication is checked by protected route middleware and API authorization. Session queries are scoped to the current user where appropriate. Passwords are hashed, security headers are enabled, allowed origins are configured, and audio upload size and format are checked. The frontend uses feature slices and page/component separation; the backend separates routes, controllers, models, middleware, and AI service logic.

Table 3. Non-functional quality requirements

Quality	Design consideration
Security	JWT protection, ownership checks, hashed password
Usability	Clear status, progress, responsive controls
Reliability	Input checks, retries, fallback, error handling
Maintainability	Feature slices and service separation

CHAPTER 10

System Architecture Overview

Layered design

The system has a browser frontend, a Node.js/Express backend, a MongoDB database, and a Python/FastAPI AI microservice. The browser handles navigation and interactive controls. The backend is the central application boundary for authentication, session ownership, data persistence, file processing orchestration, and AI provider calls. MongoDB stores durable user and session records.

Request and response path

For a new interview, the frontend sends session settings to the backend with the user's authorization token. The backend creates a pending MongoDB record and returns a session identifier. It then asks the AI service to generate questions, updates the session, and emits progress events through Socket.IO. The frontend can load the session and display questions when they are ready.

Service boundaries

The AI functionality is organized as a FastAPI service with endpoints for question generation, transcription, and evaluation. The Node backend also contains an AI integration service that calls Gemini and AssemblyAI. This separation allows AI work to be managed apart from the main web API, while the backend still enforces the application-level session and user context. Deployments must configure service URLs and provider credentials.

Figure 4. Layered architecture (conceptual)

Browser frontend	Application API	Persistent data	AI services
React / Vite Redux / Router	Express JWT / Socket.IO	MongoDB User + Session	FastAPI / Gemini AssemblyAI / FFmpeg

Requests pass through the backend, which enforces user and session rules.

CHAPTER 11

Frontend Design and Navigation

Application structure

The frontend is a React application built with Vite. React Router defines public routes for the landing, login, registration, and not-found screens, and protected routes for dashboard, profile, interview, and review pages. The shared header changes its navigation based on authentication state. Lazy loading is used for larger workspace and report pages.

State management

Redux Toolkit stores authentication and session state. The auth feature contains login, registration, Google login, and profile operations. The sessions feature contains asynchronous actions to create, retrieve, submit, end, and delete sessions. Loading, error, success, active session, and message fields let pages reflect API results without duplicating request logic.

User feedback and styling

Toast notifications communicate errors and success states. Tailwind utility classes and shared CSS create a consistent interface with dark surfaces, gradients, cards, and controls. The recent styling enhancement adds route entry animation, ambient background motion, hover lift, persistent theme selection, and reduced-motion behavior. These visual effects are secondary to the main workflow and should not interfere with task completion.

CHAPTER 12

Backend API and Request Processing

Express server

The backend uses Express 5 and mounts user and session route groups under the API prefix. Middleware configures Helmet security headers, CORS allow-listing, API rate limits, JSON/form parsing, Socket.IO, and centralized not-found and error handling. The server connects to MongoDB and runs the HTTP server with Socket.IO attached.

Authentication and session routes

User routes provide registration, login, Google authentication, and protected profile retrieval/update. Session routes apply JWT protection before their operations. The session API supports listing and creating sessions, retrieving/deleting a particular session, submitting an answer, and ending a session. Controllers validate inputs, locate session records, and verify that the current user owns a session before allowing operations.

Asynchronous work

Session creation returns an accepted response after saving an initial record. The controller performs AI question creation asynchronously, saves the resulting questions and status, and emits Socket.IO events. Answer processing similarly acknowledges receipt before transcription and evaluation finish. This approach avoids holding the browser request open for the entire model call; clients must treat status updates as part of the feature.

CHAPTER 13

Database Design and Data Dictionary

User collection

The User model stores name, unique trimmed email, password hash, optional Google identifier, and preferred role, with created/updated timestamps. The password is required for password-based accounts and hashed with bcryptjs. Google identity uses a sparse unique field. A password matching method compares entered credentials with the saved hash.

Interview session collection

A session references its owning User and stores role, level, interview type, status, scores, aggregate metrics, questions, start time, end time, and timestamps. Interview type is constrained to oral-only or coding-mix. Status is constrained to pending, in-progress, completed, or failed. The user reference is indexed to support user-specific history retrieval.

Question subdocument

Each question stores text, type, ideal answer, submitted transcript, submitted code, submitted/evaluated flags, technical score, confidence score, and AI feedback. Embedding the questions within a session keeps a session's question history together for retrieval and review. Defaults allow a newly generated question to exist before it has an answer. Sensitive submission fields are excluded from the list-history query to avoid returning them unnecessarily.

Table 4. Principal data entities

Entity	Representative fields	Relationship
User	name, email, password hash, preferredRole	Owns many sessions
Session	role, level, type, status, scores, timestamps	References one user
Question	text, type, answer, code, scores, feedback	Embedded in a session

CHAPTER 14

Authentication and Profile Management

Registration and password login

Registration validates required fields, checks for an existing email, creates a user, and returns a signed JWT with basic profile fields. Passwords are salted and hashed by the Mongoose model's save hook. Login retrieves the user and compares the entered password using bcrypt. A successful response contains the same identity information and token used by the frontend.

Google sign-in

The client obtains a Google credential and sends it to the backend. The backend verifies the ID token with the configured Google client ID and requires a verified email. If a password account already exists for that email, the Google ID can be linked; otherwise a Google-backed user record is created. This operation depends on correct OAuth client configuration in both frontend and server deployment settings.

Protected access and profile

The JWT protection middleware reads the Bearer token, verifies its signature with the configured secret, loads the corresponding user without selecting the password, and attaches that user to the request. Protected session routes use this identity to filter records and enforce ownership. The profile page allows the user to edit name and preferred role, helping the dashboard prefill the session builder.

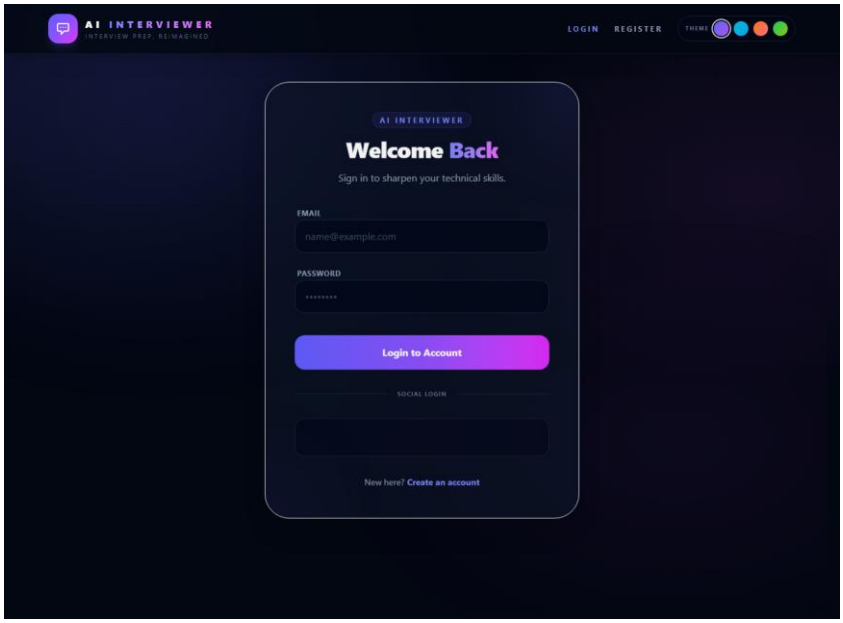


Figure 2. Login page — real application screenshot.

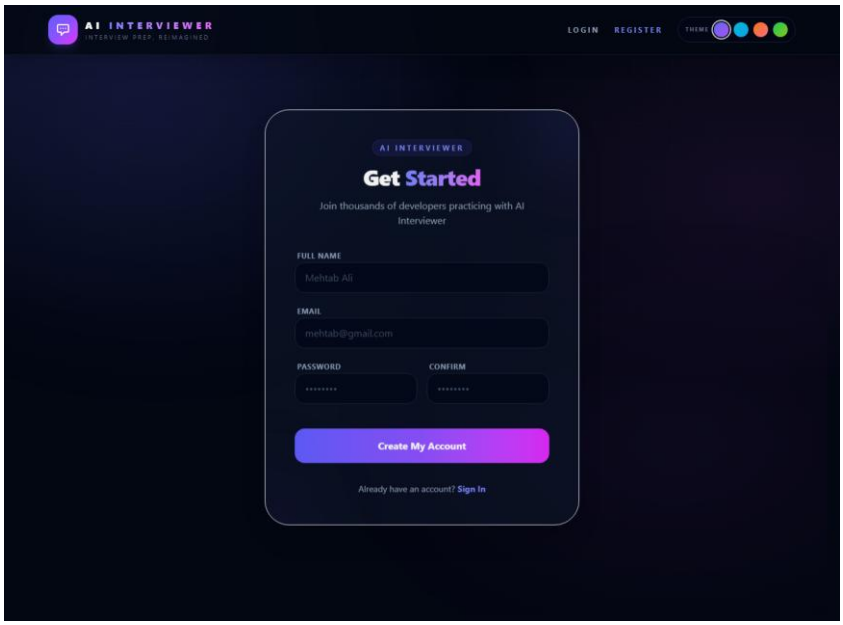


Figure 3. Registration page — real application screenshot.

CHAPTER 15

Dashboard and Session Configuration

Dashboard overview

The dashboard welcomes the authenticated user, summarizes the count of practice sessions, and presents the session builder beside interview history. Session cards show role, seniority, type, status, question count, date, and score when available. The user can resume an unfinished session, open a completed report, or delete an old entry.

Configuration controls

The session builder exposes a set of technical roles, three levels—Junior, Mid-Level, and Senior—two formats—Oral only and Coding Mix—and counts of 5, 10, or 15 questions. The preferred role from the profile is used as the initial role where available. These controlled values are submitted together so question generation has the context it needs.

Start and history behavior

When the form is submitted, Redux dispatches the create-session action and navigates to the interview route using the returned session identifier. If creation fails, a notification asks the user to retry. History is loaded for the current authenticated user. Completed sessions navigate to review, while failed sessions receive an explanatory message rather than being opened as if active.

CHAPTER 16

Question Generation Workflow

Prompt context

The question-generation service receives role, seniority, question count, and interview type. It instructs Gemini to return one question per line and avoid introductions. Oral-only sessions request conceptual questions. Coding-mix sessions request a combination, with the first portion designated for coding challenges and the rest for oral or theoretical questions.

Asynchronous lifecycle

The backend first creates a session with pending status and returns an accepted response containing the identifier. A background operation emits a generation event, calls the AI provider, maps question text to question subdocuments, saves the session as in-progress, and emits a ready event. The frontend shows a preparation state while the question list is not yet available and refreshes session details while generation remains active.

Fallback and failure handling

If AI generation fails, the session controller can create role-oriented fallback questions and record that fallback source in status messaging. If the overall session creation process cannot complete, the session is marked failed. This preserves a meaningful user-facing state instead of leaving a permanent spinner. Provider retries for transient rate limits or service errors are also used in the AI integration.

Figure 5. Question-generation lifecycle

1. Configure	2. Create	3. Generate	4. Continue
Role + level + format + count	Save pending session	Gemini or fallback	Save questions; mark in-progress

CHAPTER 17

Voice Interview and Audio Processing

Browser recording

The interview page requests microphone access through the browser's media APIs. It selects a supported recording format where possible, records audio in chunks, and displays a running duration. On stop, chunks are combined into an audio Blob and File and associated with the active question draft. Media tracks and timers are stopped when the recording ends or the page is cleaned up.

Upload and transcription

The frontend submits audio as multipart form data under the field name expected by the upload middleware, together with the question index. The backend accepts a defined set of audio formats and limits a file to 10 MB. The session controller passes the file to the transcription service, which uses AssemblyAI. Non-WAV inputs are converted to mono, 16 kHz WAV with FFmpeg where conversion succeeds.

Error recovery and cleanup

An empty audio submission is rejected before evaluation. Transcription failures are surfaced through logs and a status event, and temporary files are removed after the attempt. The interview interface can explain when transcription failed and allow a recording to be replaced. Good microphone placement and reduced background noise improve transcription quality, but the result can still contain errors and should be treated as an input to automated feedback.

Figure 6. Audio answer processing pipeline

Browser	Upload API	Transcription	Evaluation
Record audio	Validate type + size	AssemblyAI; FFmpeg WAV conversion	Save scores + feedback

CHAPTER 18

Coding Interview Workspace

Editor experience

For coding questions, the application embeds Monaco Editor, the editor technology also used by Visual Studio Code. The supported language list includes JavaScript, TypeScript, Python, Java, C++, C#, Go, Swift, Kotlin, R, SQL, HTML, CSS, Solidity, Shell, YAML, Markdown, and plain text. The application can select a default language based on the chosen role.

Draft management

Code is stored in the question draft state and persisted in local storage using a session-specific key. This means navigating between questions does not immediately discard the local draft. Draft storage is cleared when the interview is successfully finished. Users should still avoid entering sensitive personal data into code because drafts are stored in the browser on that device.

Submission and evaluation boundary

When submitted, the code and question index are sent to the backend. The session controller checks authentication, session ownership, question index, and that at least one answer input exists. The AI evaluation service receives the question, role, level, and code. The current implementation does not claim to execute untrusted code in an isolated runtime; its evaluation is based on the submitted code content and model assessment.

CHAPTER 19

AI Evaluation and Scoring

Evaluation input

The evaluation request includes the question text and type, role, seniority, verbal transcript when present, and code when present. Prompt instructions distinguish oral reasoning from coding logic. The model is asked to return a structured object containing technicalScore, confidenceScore, aiFeedback, and idealAnswer, with scores represented as integer values from 0 to 100.

Question-level persistence

After evaluation, the backend saves the transcript and code, both scores, feedback, ideal answer, and evaluated flag in the question subdocument. The session can emit a real-time update when feedback is ready. If evaluation fails, the controller attempts to reset submission/evaluation state so that the question is not falsely presented as successfully evaluated.

Session-level metrics and interpretation

For evaluated questions, the backend calculates average technical and confidence scores and derives an overall score from those averages. The review page shows these figures and question-level charting. Scores are model-generated estimates, not calibrated psychometric measurements. A candidate should read the written rationale and compare it with their own understanding instead of treating a numerical result as a guaranteed measure of job readiness.

CHAPTER 20

Session State and Real-Time Updates

Status model

The session status progresses from pending to in-progress and then completed, with failed used when generation cannot finish. Individual questions also distinguish submitted from evaluated. These states help the interface decide whether to show a preparation message, an active question, waiting feedback, or a finished report.

Socket.IO events

The server associates a connected socket with a user room using the user identifier from the connection query. Session events are emitted to that room with the session identifier, status, message, and sometimes the updated session object. The frontend socket hook listens for updates and dispatches them into Redux, allowing the user to see generation and evaluation changes without manually refreshing each time.

Why asynchronous updates matter

Question generation and audio evaluation can take longer than a basic API request. Returning an accepted response promptly keeps the initial browser interaction responsive. Real-time status events provide feedback while background work continues. The session is also retrievable over the API, which provides a recovery path if a browser temporarily misses an event or reconnects.

Table 5. Session states

State	Meaning in workflow
pending	Session record exists; questions are being prepared
in-progress	Questions are ready and answers may be submitted
completed	Evaluation is complete; report is available
failed	Question-generation or session processing could not complete

CHAPTER 21

Session Review and Progress Analytics

Summary cards

The review page displays the overall result, average technical score, average confidence score, and session duration. The duration is calculated from session start and end timestamps. These high-level values help the learner identify an overall pattern before opening the detailed answer sections.

Question-level chart

A bar chart built with Chart.js plots technical score for each question. The x-axis labels identify question order and the y-axis ranges from zero to one hundred. The chart supports quick comparison across questions, while the detailed section below preserves the context needed to interpret each score.

Detailed answer review

For every question the review page displays the prompt, technical and confidence values, submitted code or transcript when available, AI feedback, and ideal answer. This is the learning core of the report: a user can compare their attempt with the suggested answer and identify a specific concept or explanation to revisit. The review is available only after the session is completed.

Figure 7. Review report components

Summary	Chart	Answer details
Overall, technical, confidence, duration	Technical score by question	Prompt, transcript/code, AI feedback, ideal answer

CHAPTER 22

Interface Design, Themes, and Accessibility

Visual language

The interface uses a dark visual foundation with gradient accents, rounded panels, clear labels, and cards to group related work. The landing page introduces the value of the application and gives clear sign-up and sign-in actions. The shared header provides product identity and navigation, while authenticated screens emphasize the session task.

Theme choices and motion

The header provides four accent palettes: Midnight, Ocean, Sunset, and Forest. A selected palette is stored in browser local storage and applied across the interface. The app includes ambient background animation, page entry transitions, and interactive card hover effects. These effects add visual feedback while the reduced-motion media query limits animation for users who request reduced motion.

Responsive and keyboard behavior

Layouts use responsive breakpoints and a mobile navigation menu so the central workflow can adapt to smaller viewports. Form inputs have focus styling, and focus-visible outlines make keyboard position easier to identify. Accessibility can be improved further with systematic screen-reader testing, contrast review, explicit microphone permission guidance, and automated accessibility checks.

Figure 8. Theme options implemented in the interface

Midnight	Ocean	Sunset	Forest
Indigo / violet	Cyan / blue	Orange / rose	Green / lime

CHAPTER 23

Security, Privacy, and Data Handling

Identity and authorization

Password values are hashed before persistence, and passwords are not selected in protected user lookups. JWT middleware verifies the token and loads the user. Session queries check the owner before access or deletion. These checks reduce the chance that one signed-in user can retrieve another user's private interview record by changing an identifier.

Transport and request protection

The Express server enables Helmet, constrains cross-origin requests to configured origins, and applies request limits at the API and session levels. Upload middleware restricts accepted audio types and caps individual files at 10 MB. The session controller removes temporary audio files after transcription. Production deployment should use HTTPS and store credentials only in deployment environment settings.

Privacy considerations

Audio recordings and transcripts may contain personal information. A deployment should explain how such content is processed, who provides transcription and AI services, how long data is retained, and how a user can request removal. The current application persists answer text and code in its session model; users should not submit secrets or unrelated sensitive information. Privacy and data-retention policies are important future operational documentation.

CHAPTER 24

Error Handling and Reliability

Input validation

The session creation endpoint requires role, level, interview type, and count. Answer submission validates the session identifier, question index, ownership, file size, and that code or audio is present. The browser blocks an empty local answer and confirms before ending a session. These checks prevent common incomplete requests from becoming misleading successful results.

Provider and network errors

The AI service uses retry logic for selected temporary rate-limit or availability errors. When generation still fails, fallback questions may be used, and irrecoverable session creation is marked failed. Transcription and evaluation errors are logged and surfaced through status updates. User-facing toast messages provide a short action-oriented signal, while server logs are available to developers for diagnosis.

Operational caveats

Reliability depends on MongoDB and third-party services as well as application code. A client may disconnect while background work continues; the stored session can be fetched again. Rate limiting protects provider quotas but can delay repeated attempts. Future improvements should add structured logs, correlation identifiers, monitoring, queue-backed jobs, service health checks, and retry policies that distinguish safe retries from duplicate submissions.

CHAPTER 25

Testing Strategy and Test Cases

Testing approach

Testing should be organized around the user journey and service boundaries. Unit tests can verify password matching, schema constraints, score aggregation, and validation rules. API tests can use a test database and mocked AI providers. Frontend tests can verify form validation and route protection. End-to-end tests should exercise registration, session setup, answer submission, completion, and review.

Representative cases

Table entries in this report cover valid and invalid registration, correct and incorrect login, unauthenticated access, question generation success and fallback, invalid question indices, empty audio or code, allowed and rejected audio formats, a completed score summary, and ownership checks. Expected results focus on visible status, appropriate HTTP outcome, persisted session state, and absence of unauthorized data exposure.

Current validation and future coverage

The project has a production build command and frontend lint command. The frontend build succeeds in the reviewed workspace; the repository-wide lint includes issues outside the report scope, so a clean build is not equivalent to a fully passing lint suite. Automated end-to-end tests are documented as future work. Before college submission, a live demonstration should use an account and configuration that the student is authorized to access.

CHAPTER 26

Test Case Table and Expected Results

Authentication tests

TC-01: submit complete registration data and expect a created account and token. TC-02: register an already-used email and expect a clear error with no duplicate account. TC-03: use a wrong password and expect authentication failure. TC-04: request a protected session route without a valid token and expect an unauthorized response. TC-05: use a valid signed-in token to read only the current user's sessions.

Interview workflow tests

TC-06: create a session with valid role, level, format, and count; expect a pending record followed by questions and in-progress status. TC-07: simulate temporary AI failure; expect retry or fallback questions where supported. TC-08: open an invalid session identifier; expect a controlled error. TC-09: submit an empty answer; expect rejection rather than a completed evaluation. TC-10: submit a valid code answer; expect an accepted response followed by saved feedback.

Audio and report tests

TC-11: submit an allowed recording within the file size limit; expect transcription and evaluation status updates. TC-12: submit an invalid file extension or oversized file; expect rejection. TC-13: complete all questions; expect completed status, end time, and aggregate metrics. TC-14: view the report; verify summary metrics, chart, answer transcript/code, feedback, and ideal answers correspond to the same session.

Table 6. Representative test cases and expected results

ID	Test	Expected result
TC-01	Valid registration	Account created; authenticated response returned
TC-02	Duplicate registration	Request rejected; no duplicate user created
TC-03	Invalid login	Unauthorized response; error shown
TC-04	Protected route	Access denied
TC-05	Create session	Session created; questions generated or fallback used
TC-06	Invalid audio	Upload rejected with an error
TC-07	Empty answer	Submission rejected

CHAPTER 27

User Manual: Account and Session Setup

Opening the application

Open the deployed application URL or the locally configured frontend in a modern browser. The landing page explains the product and offers registration or sign-in. A new user selects registration, enters a name, email, and password, and submits the form. An existing user selects sign-in. Google sign-in is available when the OAuth client is configured and the browser completes the provider flow.

Creating an interview

After authentication, open the dashboard. Select a role that matches the target position, choose Junior, Mid-Level, or Senior, and choose Oral only or Coding Mix. Choose 5, 10, or 15 questions. Check the selections before starting, because the role and level provide context to the generated questions. The system opens the newly created session and shows a preparation message while question generation is in progress.

Profile and history

Open Profile to update the displayed name or preferred role. The dashboard history lists prior sessions and their status. Select an in-progress session to continue, or a completed session to inspect its review. A user can remove an unwanted session after confirming the delete action. Before using voice features, check browser microphone permission and test the input device.

CHAPTER 28

User Manual: Answering and Reviewing

Answering an oral question

Read the current question and progress indicator. Select the microphone control and speak clearly in a quiet environment. Stop the recording when finished and submit the answer. Wait for the processing status to finish before ending the interview. If transcription fails, the screen explains that the recording may need to be replaced; an oral answer can be affected by microphone quality, background noise, or a provider outage.

Answering a coding question

Choose the appropriate programming language in the editor and write the response. Move between questions using the question navigation controls; local code drafts are retained for the session on the same browser. Submit code when ready. The current evaluation assesses the submitted code through AI rather than claiming to run it in a secure compiler environment. Review the feedback critically and test code independently where appropriate.

Finishing and learning

After all questions have been evaluated, choose the finish action and confirm. The system calculates summary metrics and routes to the report. Review the chart, individual scores, transcript or code, feedback, and ideal answer. Record a learning note outside the app if a concept needs further study. Repeating a new session later can provide additional practice, but score changes should be interpreted cautiously.

CHAPTER 29

Installation and Configuration

Prerequisites

Local development requires Node.js and npm for the frontend and backend, Python for the FastAPI service, MongoDB availability, and provider credentials for AI and transcription features. FFmpeg is included through a backend package for audio conversion. The project repository contains separate frontend, backend, and AI service folders, along with deployment configuration.

Starting the services

For the AI service, create and activate a Python virtual environment, install requirements, and run the service entry point. In the backend folder, install npm dependencies and run the development script. In the frontend folder, install npm dependencies and run Vite. The README documents Windows activation syntax as well as the separate service steps. Each service should be started in its own terminal.

Environment variables

The AI service expects its port and Gemini key. The backend requires environment mode, port, MongoDB URI, JWT secret, AI service URL, AssemblyAI key, and Gemini key; OAuth configuration is needed when using Google login. The frontend requires API and Socket.IO URLs and a Google client ID. Use local ignored environment files or deployment secret settings, and never commit real credentials.

Table 7. Environment variable categories

Service	Configuration purpose
Frontend	API URL, Socket.IO URL, Google OAuth client ID
Backend	Port, database URI, JWT secret, provider keys, AI URL
AI service	Service port and Gemini key

CHAPTER 30

Deployment and Operations

Deployment topology

The project README describes a Vercel deployment for the frontend and Render for backend and AI services. A render.yaml file is present for service configuration. The backend uses an allow-list containing the deployed frontend origin and development origins. Deployed environment variables must match the service URLs and credentials used by the frontend and backend.

Release checklist

Before release, verify production API URLs, OAuth client configuration, CORS origins, database connectivity, JWT secret strength, provider credentials, audio upload size, and service health. Run a production frontend build. Test account creation, session creation, one supported answer type, and report rendering against the deployed environment. Inspect logs for provider and upload failures without exposing tokens or personal data.

Ongoing operation

Operational ownership includes monitoring API errors, AI provider limits, database growth, storage of transcripts, and deletion behavior. Rate limits protect the backend and AI quota. A deployment should have a process for rotating exposed credentials, updating dependencies, reviewing security advisories, and communicating retention practices. The README already includes a security checklist reminding maintainers to keep environment files ignored and review production configuration.

CHAPTER 31

Limitations and Known Constraints

AI limitations

AI-generated questions and scores can be inconsistent, incomplete, or biased by prompt and model behavior. A score does not constitute a validated hiring assessment. Ideal answers may be one reasonable solution rather than the only correct one. Human review and independent practice remain important, particularly for complex architecture or coding questions.

Service and device constraints

Voice practice requires a microphone, browser permission, supported audio encoding, network access, and external transcription availability. Poor recordings can reduce transcript quality. AI processing introduces latency and may encounter rate limits. Local question drafts are browser-specific; they should not be assumed to synchronize between devices unless the server has received the submitted response.

Scope and validation constraints

The current project does not include automated end-to-end coverage for every workflow, a background queue for scaling high-volume jobs, a longitudinal analytics dashboard, or a mock mode. The code review and frontend build provide useful checks but do not replace integration testing with production credentials. Reported configuration and deployment behavior should be confirmed in the environment used for the final demonstration.

CHAPTER 32

Future Scope and Enhancements

Testing and quality improvements

A priority is automated end-to-end testing for authentication and complete voice interviews, as identified in the project plan. Additional coverage could include code submissions, provider errors, session ownership, accessibility, browser compatibility, and mobile layouts. AI calls should be mockable so that tests are deterministic and do not consume provider quota.

Scalability and analytics

Background job queues can make long question-generation and evaluation operations more robust under high request volume. Queue retry and idempotency rules would prevent duplicate evaluation. A long-term analytics dashboard could visualize score trends by role, level, question type, and time, while clearly explaining that scores are AI estimates. Caching carefully selected templates may reduce redundant calls.

Learning and accessibility

A mock/demo mode would allow students or evaluators to explore the interface without external AI credentials or quota use. Future learning features could include user-controlled notes, recommended topics, adjustable feedback detail, multilingual transcription, and accessibility testing. Any additional data should be purpose-limited, transparent, and protected with an explicit retention policy.

Table 8. Future enhancement roadmap

Priority area	Potential enhancement
Quality	Automated end-to-end tests and accessibility checks
Scalability	Background queue, retries, and idempotent jobs
Learning	Progress analytics, topic insights, and mock mode

CHAPTER 33

Conclusion

Project outcome

AI Interviewer demonstrates a full-stack interview-practice workflow. Users can register or sign in, configure a role-focused session, receive AI-generated questions, answer through voice or code, and review question-level feedback and scores. The system combines a React/Vite frontend, an Express API, MongoDB models, a Python/FastAPI AI service, and external Gemini and AssemblyAI services.

Learning achieved

The implementation illustrates practical software engineering concerns beyond rendering a page: JWT authentication, ownership checks, data modelling, file uploads, audio conversion, asynchronous work, real-time events, structured model output, error states, deployment variables, and responsive UI. Integrating such components required coordinating request formats and state transitions across multiple services.

Final remarks

The application is a practice aid with clear extension opportunities. Its AI feedback should be interpreted responsibly and not as a guaranteed measure of employment readiness. With more automated testing, queue-based processing, analytics, mock mode, and privacy documentation, the platform can become easier to maintain and demonstrate. The central project goal—making interview practice structured and reviewable—is delivered through the current workflow.

REFERENCES

[1] AI Interviewer project repository, README.md, setup instructions, project phases, security checklist, and deployment notes.

[2] React documentation, https://react.dev/ — component-based user interface library.

[3] Vite documentation, https://vite.dev/ — frontend development and build tool.

[4] Redux Toolkit documentation, https://redux-toolkit.js.org/ — Redux application state management.

[5] Express documentation, https://expressjs.com/ — Node.js web framework.

[6] Mongoose documentation, https://mongoosejs.com/ — MongoDB object modelling for Node.js.

[7] MongoDB documentation, https://www.mongodb.com/docs/ — document database concepts.

[8] FastAPI documentation, https://fastapi.tiangolo.com/ — Python API framework.

[9] Google AI for Developers, https://ai.google.dev/gemini-api/docs — Gemini API documentation.

[10] AssemblyAI documentation, https://www.assemblyai.com/docs/ — audio transcription API.

[11] FFmpeg documentation, https://ffmpeg.org/documentation.html — media conversion and processing.

[12] Socket.IO documentation, https://socket.io/docs/v4/ — bidirectional event-based communication.

[13] Monaco Editor, https://microsoft.github.io/monaco-editor/ — browser code editor.

[14] Chart.js documentation, https://www.chartjs.org/docs/ — charts used for question-level review.

[15] Tailwind CSS documentation, https://tailwindcss.com/docs/ — utility-first styling framework.

All web references were accessed for project documentation and are listed as implementation references. Provider behavior, prices, and API specifications may change over time; consult the current official documentation when maintaining or deploying the system.

APPENDIX A — GLOSSARY

Term	Explanation
Asynchronous processing	Work that continues after an API request has returned an initial acknowledgement.
Confidence score	One of the AI-generated score fields stored for an evaluated answer; it is an estimate, not a validated psychological measurement.
Fallback question	A role-oriented question produced by backend fallback logic if AI generation fails.
JWT	A signed token used by the backend to verify an authenticated user's identity.
Question subdocument	A question record embedded within its parent interview session document.
Socket room	A Socket.IO grouping used to emit events to connections associated with a user.
Transcription	Conversion of recorded speech into text for use in answer evaluation.

APPENDIX B — DEMONSTRATION CHECKLIST

[ ] Confirm frontend URL and backend services are available.

[ ] Use a test account; do not display a personal password during the demonstration.

[ ] Check microphone permission and input volume before class.

[ ] Select a known role, seniority, format, and short question count.

[ ] Explain that question generation can take time because it is asynchronous.

[ ] Submit one short spoken or code-based answer and wait for status updates.

[ ] Open the completed review and explain score, chart, transcript/code, and feedback.

[ ] If the network or provider is unavailable, explain the workflow using the report and do not expose credentials.

APPENDIX C — PROJECT CONFIGURATION CHECKLIST

Check	Purpose	Completed
MongoDB URI	Backend can connect to the configured database	[ ]
JWT secret	Sign and verify user access tokens	[ ]
Gemini key	Question generation and evaluation	[ ]
AssemblyAI key	Audio transcription	[ ]
AI service URL	Backend can reach the AI service	[ ]
Frontend API URL	Browser sends requests to the right backend	[ ]
OAuth client IDs	Google sign-in matches deployed origins	[ ]
CORS allow-list	Permit only intended frontend origins	[ ]

This checklist is for deployment preparation only. Actual secret values should be configured in environment settings and must not be written into the college report, screenshots, source code, or a public repository.